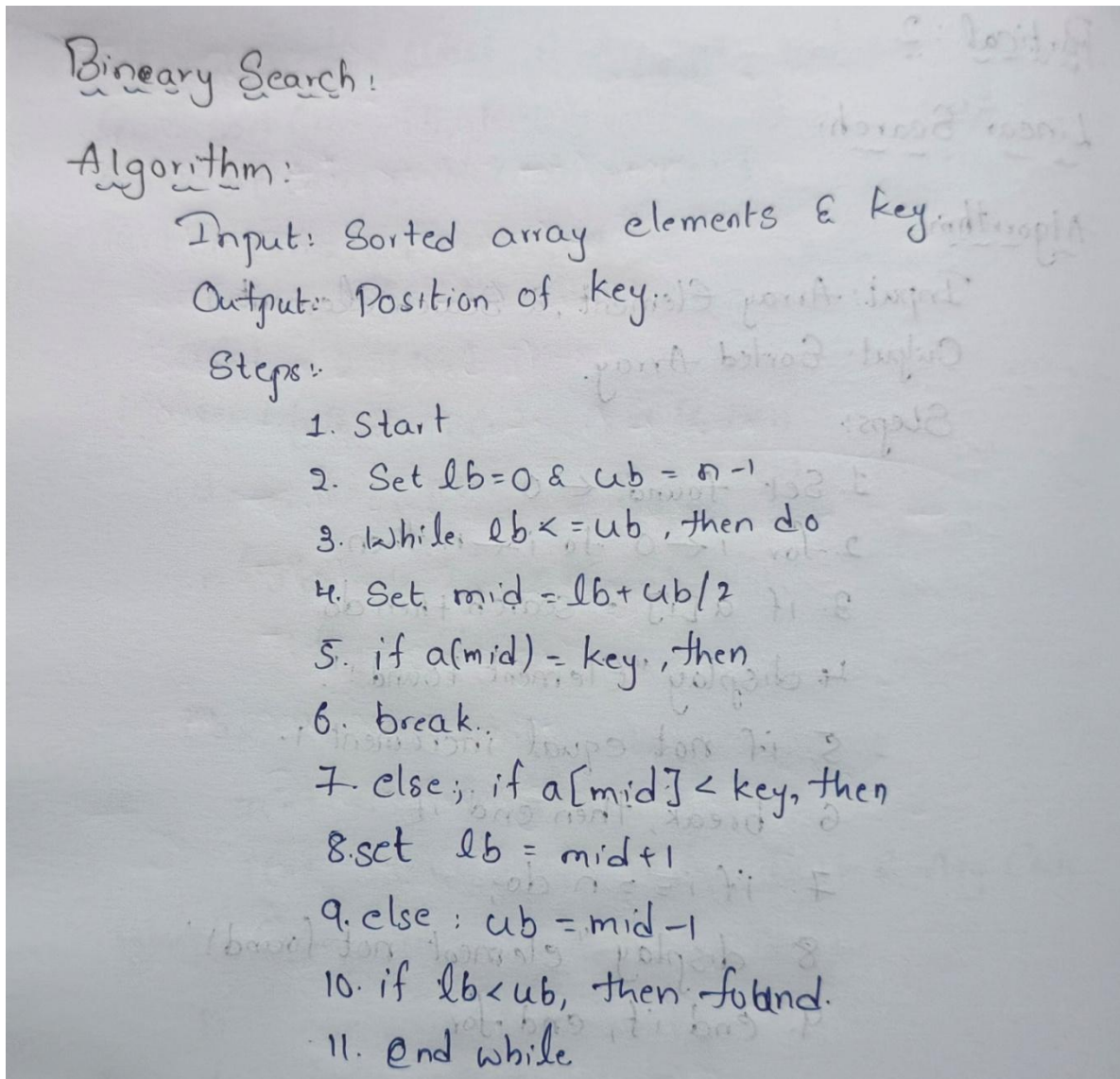




PRACTICAL - 2

AIM : Implementation and Time analysis of linear and binary search algorithm

Binary Search Algorithm :





Code :

```
#include <iostream> using namespace
std;

int main() {    int n, arr[100],
key;
    int low, high, mid;

    cout << "Enter size of array: ";
    cin >> n;

    cout << "Enter sorted array elements: ";
    for(int i = 0; i < n; i++)
    cin >> arr[i];

    cout << "Enter element to search: ";
    cin >> key;

    low = 0;
    high = n - 1;

    while(low <= high)
    {
        mid = (low + high)
/ 2;

        if(arr[mid] == key)
        {
            cout << "Element found at position " << mid + 1;
return 0;
        }
        else if(arr[mid] < key)
        {
            low = mid + 1;
        }
        else
        {
            high = mid - 1;
        }
    }
}
```



```
    }  
}  
  
cout << "Element not found";  
  
return 0;  
}
```

OUTPUT :

```
● PS C:\Users\prash\Downloads\DAA Lab> cd "c:\Users\prash\Downloads\DAA Lab\" ; if  
  ($?) { g++ binarysearch.cpp -o binarysearch } ; if ($?) { .\binarysearch }  
Enter number of elements: 5  
Enter 5 elements in sorted order: 34 67 98 12 67  
Enter element to search: 98  
Element found at position: 3  
○ PS C:\Users\prash\Downloads\DAA Lab> |
```

TIME COMPLEXITY :

best case & average case & worst case:

Time Complexity: Best Case:

```
for (i=0; i<n; i++) {  
    if (a[i] == k) {  
        return i;  
    }  
}
```

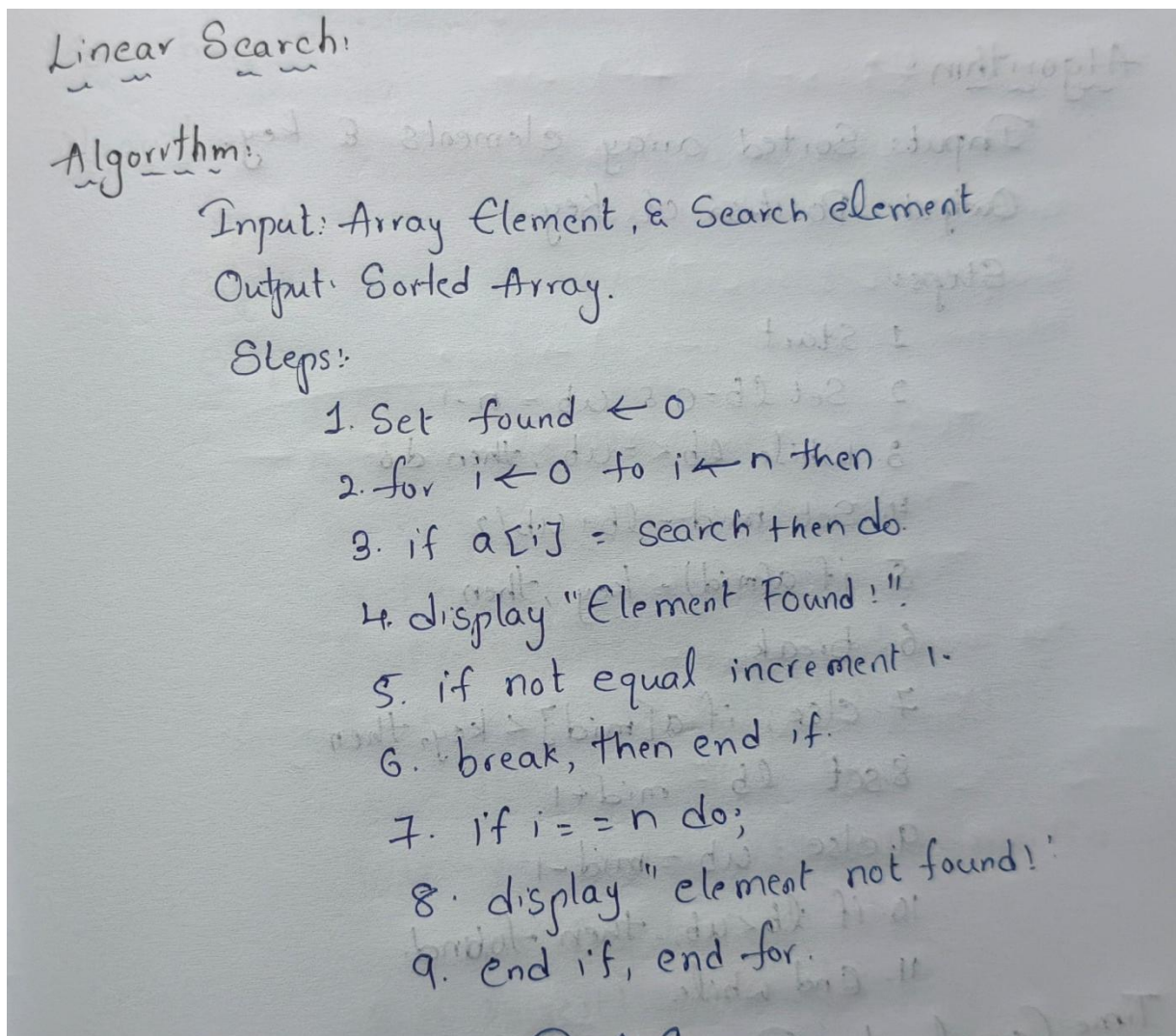
$T(n) = 1$
 $\therefore T(n) = O(1) \rightarrow$ Best Case

Average & Worst Case:

$\rightarrow \frac{n+1}{2}$
 $T(n) = \frac{n+1}{2} \Rightarrow T(n) = O(n)$
 $T(n) = \frac{n}{2} + \frac{1}{2}$



Linear Search Algorithm :



Code :

```
#include <iostream>
using namespace std;

int main() {
    int n, arr[100], key;

    cout << "Enter size of array: ";
    cin >> n;

    cout << "Enter array elements: ";
    for(int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Enter element to search: ";
    cin >> key;

    for(int i = 0; i < n; i++)
    {
        if(arr[i] == key)
        {
            cout << "Element found at position " << i + 1;
            return 0;
        }
    }

    cout << "Element not found";

    return 0;
}
```

OUTPUT :

```

Element found at position: 3
PS C:\Users\prash\Downloads\DAA Lab> cd "c:\Users\prash\Downloads\DAA Lab\" ; if
($?) { g++ linearsearch.cpp -o linearsearch } ; if ($?) { .\linearsearch }
Enter number of elements: 7
Enter 7 elements: 34 67 89 53 12 47 78
Enter element to search: 53
Element found at position: 4
PS C:\Users\prash\Downloads\DAA Lab>

```

TIME COMPLEXITY :

best case & average case & worst case:

Time Complexity: Best Case:
 $mid = (lb + ub) / 2 \rightarrow 1$
 $if (a[mid] == Key) \rightarrow 1$
 $T(n) = 1$
 $T(n) = O(1)$

Average & Worst Case:
 After k division: $\frac{n}{2^k} = 1$
 $\therefore n = 2^k$
 $k = \log_2 n$
 $\therefore T(n) = O(\log n)$

n
 $\downarrow$
 $n/2$
 $\downarrow$
 $n/4$
 $\vdots$

Conclusion :

Linear Search and Binary Search were successfully implemented. Linear Search checks elements one by one, so it is simple but slower for large lists. Binary Search divides the list into two parts repeatedly, making it faster, but the list must be sorted. Therefore, Binary Search is faster, while Linear Search is simpler and works on unsorted lists.